

컴퓨터 네트워크 과제 [선택 1] TCP Socket HTTP Server/Client

Python TCP Socket 기반 HTTP Request/Response 및 파일 제어 구현 보고서

국민대학교 소프트웨어학부 20233044 박동주

환경: macOS, localhost 127.0.0.1, port 8080 | 캡처: Wireshark lo0

목차

1. 개요
2. 구현 목적
3. 개발 환경
4. 사용 기술
5. TCP Socket 통신 흐름
6. HTTP Request 메시지 구조
7. HTTP Response 메시지 구조
8. 구현한 Method-Response Case
9. server.py 코드 설명
10. client.py 코드 설명
11. 실행 방법
12. Client / WireShark 실행결과
13. Server 실행 결과
14. 전체 실행 결과 요약

1. 개요

본 과제는 Python으로 TCP 기반 소켓 Server와 Client 프로그램을 작성하고, Client가 HTTP 형식의 Request를 직접 생성하여 Server로 전송하는 프로젝트이다.

Server는 Python socket 모듈만 사용하여 TCP 연결을 수락하고, Request Line을 파싱하여 Method, Path, Version을 분리한다. 이후 GET, HEAD, POST, PUT, DELETE, 잘못된 Method 요청에 따라 HTTP Response를 직접 구성한다.

실행 환경은 macOS, localhost 127.0.0.1, port 8080을 기준으로 하며, Wireshark에서는 loopback 인터페이스인 lo0에서 tcp.port == 8080 필터로 패킷을 캡처한다.

2. 구현 목적

웹 프레임워크 없이 TCP Socket만으로 HTTP 메시지 흐름을 구현하여 응용 계층 프로토콜이 실제 네트워크 위에서 어떻게 동작하는지 이해하는 것이 목적이다.

GET은 파일 조회, POST는 파일 생성 및 append, PUT은 파일 덮어쓰기, DELETE는 파일 삭제 기능과 연결하였다. 따라서 단순한 문자열 응답이 아니라 실제 파일 제어를 수행하는 Server 중심 구현이다.

3. 개발 환경

운영체제: macOS

언어: Python 3

통신 방식: TCP Socket

주소 및 포트: 127.0.0.1:8080

캡처 도구: Wireshark

4. 사용 기술

socket.AF_INET은 IPv4 주소 체계를 의미하고 socket.SOCK_STREAM은 TCP 스트림 소켓을 의미한다.

Server는 bind(), listen(), accept(), recv(), sendall()을 직접 사용한다. Client도 TCP 연결 후 sendall()과 recv()로 HTTP 형식 메시지를 주고받는다.

Response는 HTTP/1.1 status line, Date, Server, Content-Type, Content-Length, Connection header, blank line, body 순서로 직접 작성한다.

5. TCP Socket 통신 흐름

Server는 `socket()`으로 TCP 소켓을 만들고 `bind()`로 127.0.0.1:8080에 연결한다.

`listen()`으로 연결 요청을 기다린 후 `accept()`로 Client 연결을 수락한다.

Client는 `socket()`으로 TCP 소켓을 만들고 `connect()`로 Server에 접속한다.

Client는 `sendall()`로 HTTP Request를 전송하고, Server는 `recv()`로 Request를 수신한다.

Server는 처리 결과를 `sendall()`로 전송하고, Client는 `recv()`를 반복하여 Response 전체를 출력한다.

6. HTTP Request 메시지 구조

Request Line 예시: GET /index.html HTTP/ 1.1

Header에는 Host, User-Agent, Content-Type, Content-Length, Connection을 포함한다.

Header와 Body 사이에는 반드시 blank line이 있어야 하며, 이 프로젝트에서는 CRLF 문자 조합을 사용하여 HTTP 형식에 맞춘다.

POST와 PUT은 Body를 입력받아 Content-Length를 byte 길이로 계산한 뒤 전송한다.

7. HTTP Response 메시지 구조

Response Status Line 예시: HTTP/ 1.1 200 OK

Server는 Date, Server, Content-Type, Content-Length, Connection header를 직접 구성한다.

GET, POST, DELETE 오류 응답은 Body를 포함할 수 있고, HEAD와 204 No Content 응답은 Body 없이 header 중심으로 응답한다.

Wireshark에서는 이 구조가 HTTP format으로 해석되어 Request Line과 Status Line을 확인할 수 있다.

8. 구현한 Method-Response Case

No	Request	Response	기능
1	GET /index.html	200 OK	HTML 파일 조회
2	GET /notfound.html	404 Not Found	없는 파일 오류
3	HEAD /index.html	200 OK	Body 없이 Header만 응답
4	HEAD /missing.html	404 Not Found	없는 파일에 대해 Header만 응답
5	POST /post.txt	201 Created	파일 생성
6	POST /post.txt	200 OK	파일 append
7	PUT /put.txt	204 No Content	파일 덮어쓰기
8	PUT /readonly.txt	403 Forbidden	수정 금지
9	DELETE /post.txt	200 OK	파일 삭제
10	DELETE /readonly.txt	403 Forbidden	삭제 금지
11	PATCH /index.html	400 Bad Request	잘못된 Method

GET /index.html -> 200 OK: index.html 파일 조회 성공

GET /notfound.html -> 404 Not Found: 없는 파일 요청

HEAD /index.html -> 200 OK: Body 없이 header만 응답

HEAD /missing.html -> 404 Not Found: 없는 파일에 대한 HEAD 응답

POST /post.txt -> 201 Created: 파일이 없으면 새로 생성

POST /post.txt -> 200 OK: 파일이 있으면 기존 내용 뒤 append

PUT /put.txt -> 204 No Content: 파일 내용 덮어쓰기

PUT /readonly.txt -> 403 Forbidden: 수정 금지 처리

DELETE /post.txt -> 200 OK: 파일 삭제 성공

DELETE /readonly.txt -> 403 Forbidden: 삭제 금지 처리

PATCH /index.html -> 400 Bad Request: 지원하지 않는 Method 처리

9. server.py 코드 설명

9-1. 기본 설정 (Line 1~12)

```
1 import datetime as dt
2 import socket
3 from pathlib import Path
4 from typing import Dict, Optional, Tuple
5 from urllib.parse import unquote
```

line 1~5 : TCP 소켓 통신, 날짜 처리, 파일 경로 처리, 타입 힌트, URL 디코딩을 위한 모듈을 import한다.

```
8 HOST = "127.0.0.1"
9 PORT = 8080
10 BUFFER_SIZE = 4096
11 SERVER_NAME = "PythonSocketHTTPServer/1.0"
12 BASE_DIR = Path(__file__).resolve().parent
```

line 8~12 : 서버 실행에 필요한 기본 값을 설정한다. HOST는 localhost 주소인 127.0.0.1, PORT는 8080으로 지정하였다. BUFFER_SIZE는 TCP 데이터 수신 크기이고, SERVER_NAME은 HTTP Response Header의 Server 값으로 사용된다. BASE_DIR은 서버가 파일을 읽고 쓰는 기준 폴더이다.

9-2. HTTP 날짜 및 Content-Type 처리 (Line 15~26)

```
15 def make_http_date() -> str:
16     """HTTP Response의 Date 헤더에 들어갈 GMT 시간 문자열을 만든다."""
17     return dt.datetime.now(dt.timezone.utc).strftime("%a, %d %b %Y %H:%M:%S GMT")
18
19
20 def guess_content_type(path: str) -> str:
21     """요청 경로의 확장자에 따라 간단한 Content-Type을 결정한다."""
22     if path.endswith(".html"):
23         return "text/html; charset=utf-8"
24     if path.endswith(".txt"):
25         return "text/plain; charset=utf-8"
26     return "text/plain; charset=utf-8"
```

make_http_date() 함수는 HTTP Response Header의 Date 값을 GMT 형식으로 생성한다. HTTP 응답 메시지는 Date Header를 포함해야 하므로, 서버가 응답을 만들 때마다 현재 시간을 문자열로 변환하여 사용한다.

guess_content_type() 함수는 요청받은 파일 확장자에 따라 Content-Type을 결정한다. .html 파일은 text/html; charset=utf-8, .txt 파일은 text/plain; charset=utf-8로 응답하도록 구현하였다.

9-3. HTTP Request 파싱 (Line 29~52)

parse_http_request() 함수는 TCP로 수신한 bytes 데이터를 UTF-8 문자열로 변환한 뒤 HTTP Request 구조에 맞게 분리한다. HTTP Request는 Request Line, Header, Blank Line, Body 구조를 가진다.

```
37 request_text = raw_data.decode("utf-8", errors="replace")
38 header_part, _, body = request_text.partition("\r\n\r\n")
39 lines = header_part.split("\r\n")
```

line 37~38 : 수신한 bytes 데이터를 문자열로 디코딩하고, \r\n\r\n을 기준으로 Header 부분과 Body 부분을 분리한다.

```
41 if not lines or len(lines[0].split()) != 3:
42     return "", "", {}, body
43
44 method, path, version = lines[0].split()
```

line 41~44 : 첫 번째 줄인 Request Line에서 Method, Path, Version을 추출한다. 예를 들어 GET /index.html HTTP/ 1.1 요청이 들어오면 Method는 GET, Path는 /index.html, Version은 HTTP/ 1.1로 분리된다.

```
45     headers: Dict[str, str] = {}
46
47     for line in lines[1:]:
48         if ":" in line:
49             key, value = line.split(":", 1)
50             headers[key.strip().lower()] = value.strip()
51
52     return method.upper(), path, version, headers, body
```

line 45~52 : Header 영역의 각 줄을 key: value 형태로 분석하여 딕셔너리에 저장한다. 이 과정에서 Host, User-Agent, Content-Length, Connection 등의 값을 확인할 수 있다.

9-4. HTTP Response 메시지 구성 (Line 55~86)

build_http_response() 함수는 Server가 Client에게 보낼 HTTP Response 메시지를 직접 생성한다. HTTP Response는 Status Line, Header, Blank Line, Body 구조를 가진다.

```
72     response_lines = [
73         f"HTTP/1.1 {status_code} {reason}",
74         f"Date: {make_http_date()}",
75         f"Server: {SERVER_NAME}",
76         f"Content-Type: {content_type}",
77         f"Content-Length: {len(body_bytes)}",
78         "Connection: close",
79         "",
80         "",
81     ]
```

line 72~81 : Status Line과 Header를 직접 문자열로 구성한다. 응답에는 Date, Server, Content-Type, Content-Length, Connection Header가 포함된다.

```
83     header_bytes = "\r\n".join(response_lines).encode("utf-8")
84     if send_body:
85         return header_bytes + body_bytes
86     return header_bytes
```

line 83~86 : Header 뒤에 Blank Line을 추가하고, 필요한 경우 Body를 함께 붙여 bytes 형태로 반환한다. HEAD 요청이나 204 No Content 응답처럼 Body가 없어야 하는 경우에는 Header만 전송하도록 처리하였다.

9-5. 안전한 파일 경로 처리 (Line 89~105)

```
96     clean_path = unquote(request_path).lstrip("/")
97     target_path = (BASE_DIR / clean_path).resolve()
98
99     # ../ 같은 경로 이동 공격을 막기 위해 프로젝트 폴더 내부 파일만 허용한다.
100     try:
101         target_path.relative_to(BASE_DIR)
102     except ValueError:
103         return None
104
105     return target_path
```

safe_file_path() 함수는 Client가 요청한 URL 경로를 실제 프로젝트 폴더 내부 파일 경로로 변환한다. 예를 들어 /index.html 요청은 서버 폴더 안의 index.html 파일로 변환된다.

또한 ../와 같은 상위 경로 이동을 막아 서버 폴더 밖의 파일에 접근하지 못하도록 처리하였다. 이 부분은 GET, POST, PUT, DELETE에서 실제 파일을 읽고 쓰기 전에 공통적으로 사용된다.

9-6. GET 요청 처리 (Line 108~119)

```
108 def handle_get(path: str) -> bytes:
109     """GET 요청 처리: 파일이 있으면 200 OK, 없으면 404 Not Found."""
110     file_path = safe_file_path(path)
111     if file_path is None:
112         return build_http_response(400, "Bad Request", "Invalid file path.")
113
114     if file_path.exists() and file_path.is_file():
115         body = file_path.read_text(encoding="utf-8")
116         return build_http_response(200, "OK", body, guess_content_type(path))
117
118     body = f"404 Not Found: {path} does not exist on this server."
119     return build_http_response(404, "Not Found", body)
```

handle_get() 함수는 Client가 요청한 파일을 읽어 HTTP Response로 반환한다. 파일이 존재하면 파일 내용을 읽어 200 OK 상태 코드와 함께 응답하고, 파일이 존재하지 않으면 404 Not Found 상태 코드와 오류 메시지를 응답한다.

이를 통해 GET /index.html -> 200 OK, GET /notfound.html -> 404 Not Found 케이스를 처리한다.

9-7. HEAD 요청 처리 (Line 122~138)

```
122 def handle_head(path: str) -> bytes:
123     """
124     HEAD 요청 처리: GET과 같은 상태 코드를 만들지만 Body는 전송하지 않는다.
125
126     HTTP에서 HEAD는 응답 헤더만 확인할 때 사용한다. 따라서 Content-Length는
127     실제 Body 길이를 알려주되, blank line 뒤의 Body 데이터는 보내지 않는다.
128     """
129     file_path = safe_file_path(path)
130     if file_path is None:
131         return build_http_response(400, "Bad Request", "Invalid file path.", send_body=False)
132
133     if file_path.exists() and file_path.is_file():
134         body = file_path.read_text(encoding="utf-8")
135         return build_http_response(200, "OK", body, guess_content_type(path), send_body=False)
136
137     body = f"404 Not Found: {path} does not exist on this server."
138     return build_http_response(404, "Not Found", body, send_body=False)
```

handle_head() 함수는 GET과 같은 방식으로 파일 존재 여부를 판단하지만 Body는 전송하지 않는다. 파일이 존재하면 200 OK Header만 응답하고, 파일이 없으면 404 Not Found Header만 응답한다.

HTTP에서 HEAD Method는 실제 Body를 받지 않고 Header 정보만 확인할 때 사용되므로, send_body=False로 처리하였다.

9-8. POST 요청 처리 및 파일 생성/추가 (Line 141~169)


```

141 def handle_post(path: str, body: str) -> bytes:
142     """
143     POST 요청 처리.
144
145     /post.txt 파일이 없으면 새로 만들고 201 Created를 응답한다.
146     /post.txt 파일이 이미 있으면 기존 내용 뒤에 append하고 200 OK를 응답한다.
147     """
148     if path != "/post.txt":
149         return build_http_response(404, "Not Found", "POST target path is not supported.")
150
151     file_path = safe_file_path(path)
152     if file_path is None:
153         return build_http_response(400, "Bad Request", "Invalid file path.")
154
155     file_exists = file_path.exists()
156     file_path.parent.mkdir(parents=True, exist_ok=True)
157
158     mode = "a" if file_exists else "w"
159     with file_path.open(mode, encoding="utf-8") as file:
160         if file_exists:
161             file.write("\n")
162             file.write(body)
163
164     if file_exists:
165         response_body = "POST success: post.txt already existed, so the body was appended."
166         return build_http_response(200, "OK", response_body)
167
168     response_body = "POST success: post.txt was created."
169     return build_http_response(201, "Created", response_body)

```

handle_post() 함수는 /post.txt 경로에 대한 POST 요청을 처리한다. /post.txt 파일이 존재하지 않으면 새 파일을 생성하고 Client가 보낸 Body 내용을 저장한 뒤 201 Created를 응답한다.

반대로 /post.txt 파일이 이미 존재하면 기존 내용 뒤에 Body를 추가하고 200 OK를 응답한다. 이 기능은 Client 요청에 따라 서버 쪽 파일 상태가 실제로 변경된다는 점에서 중요하다.

9-9. PUT 요청 처리 및 파일 덮어쓰기 (Line 172~191)

```

172 def handle_put(path: str, body: str) -> bytes:
173     """
174     PUT 요청 처리.
175
176     /put.txt는 파일 내용을 덮어쓰고 204 No Content를 응답한다.
177     /readonly.txt는 수정할 수 없는 파일로 가정하고 403 Forbidden을 응답한다.
178     """
179     if path == "/readonly.txt":
180         return build_http_response(403, "Forbidden", "PUT denied: readonly.txt cannot be modified.")
181
182     if path == "/put.txt":
183         file_path = safe_file_path(path)
184         if file_path is None:
185             return build_http_response(400, "Bad Request", "Invalid file path.")
186
187         file_path.parent.mkdir(parents=True, exist_ok=True)
188         file_path.write_text(body, encoding="utf-8")
189         return build_http_response(204, "No Content", "", send_body=False)
190
191     return build_http_response(404, "Not Found", "PUT target path is not supported.")

```

handle_put() 함수는 Client가 보낸 Body 내용으로 서버 파일을 덮어쓴다. PUT /put.txt 요청이 들어오면 put.txt 파일을 생성하거나 기존 내용을 덮어쓴 뒤 204 No Content를 응답한다.

204 No Content는 요청 처리는 성공했지만 Response Body를 보내지 않는다는 의미이다. PUT /readonly.txt 요청은 수정 금지 대상으로 처리하여 403 Forbidden을 응답한다.

9-10. DELETE 요청 처리 및 파일 삭제 (Line 194~216)

```

202     if path == "/readonly.txt":
203         return build_http_response(403, "Forbidden", "DELETE denied: readonly.txt cannot be removed.")
204
205     if path not in ("/post.txt", "/put.txt"):
206         return build_http_response(404, "Not Found", "DELETE target path is not supported.")
207
208     file_path = safe_file_path(path)
209     if file_path is None:
210         return build_http_response(400, "Bad Request", "Invalid file path.")
211
212     if file_path.exists() and file_path.is_file():
213         file_path.unlink()
214         return build_http_response(200, "OK", f"DELETE success: {path} was removed.")
215
216     return build_http_response(404, "Not Found", f"DELETE failed: {path} does not exist.")

```


handle_delete() 함수는 서버에 존재하는 파일을 삭제하는 기능을 수행한다. /post.txt 또는 /put.txt 파일이 존재하면 파일을 삭제하고 200 OK를 응답하며, 파일이 존재하지 않으면 404 Not Found를 응답한다.

/readonly.txt는 삭제 금지 대상으로 처리하여 403 Forbidden을 응답한다. 이 기능은 교수님 메모에서 강조한 파일 제어 동작을 보여주기 위해 추가하였다.

9-11. Method 분기 처리 (Line 219~235)

```
219 def route_request(method: str, path: str, version: str, body: str) -> bytes:
220     """파싱된 Method와 Path에 따라 알맞은 HTTP Response를 선택한다."""
221     if version != "HTTP/1.1":
222         return build_http_response(400, "Bad Request", "Only HTTP/1.1 is supported.")
223
224     if method == "GET":
225         return handle_get(path)
226     if method == "HEAD":
227         return handle_head(path)
228     if method == "POST":
229         return handle_post(path, body)
230     if method == "PUT":
231         return handle_put(path, body)
232     if method == "DELETE":
233         return handle_delete(path)
234
235     return build_http_response(400, "Bad Request", f"Unsupported method: {method}")
```

route_request() 함수는 파싱된 Method와 Path를 기준으로 실제 처리 함수를 선택한다. GET, HEAD, POST, PUT, DELETE는 각각의 handler 함수로 분기된다.

HTTP Version이 HTTP/1.1이 아니면 400 Bad Request를 응답한다. 또한 PATCH처럼 서버에서 지원하지 않는 Method가 들어오면 400 Bad Request를 응답하도록 구현하였다.

9-12. TCP Request 전체 수신 (Line 238~273)

```
while len(body_bytes) < content_length:
    chunk = client_socket.recv(BUFFER_SIZE)
    if not chunk:
        break
    body_bytes += chunk
```

receive_full_request() 함수는 TCP가 스트림 방식이라는 점을 고려하여 Request 전체를 수신한다. TCP에서는 한 번의 recv() 호출로 전체 HTTP 메시지가 모두 도착한다는 보장이 없다.

따라서 먼저 Header의 끝을 의미하는 \r\n\r\n이 나올 때까지 데이터를 수신한다. 그 다음 Header에 포함된 Content-Length 값을 읽고, Body 길이가 부족하면 추가로 recv()를 수행한다.

9-13. 서버 실행 및 로그 출력 (Line 276~317)

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

try:
    server_socket.bind((HOST, PORT))
    server_socket.listen(5)
    print(f"[SERVER] Listening on http://{HOST}:{PORT}")
    print("[SERVER] Press Ctrl+C to stop.")

    while True:
        client_socket, client_address = server_socket.accept()
        with client_socket:
            raw_request = receive_full_request(client_socket)
            request_text = raw_request.decode("utf-8", errors="replace")
            method, path, version, headers, body = parse_http_request(raw_request)
```

```
except KeyboardInterrupt:
    print("\n[SERVER] KeyboardInterrupt received. Server stopped.")
finally:
    server_socket.close()
```

run_server() 함수는 TCP 기반 HTTP Server를 실행하는 핵심 함수이다. socket.AF_INET, socket.SOCK_STREAM을 사용하여 IPv4 기반 TCP 소켓을 생성한다.

bind()로 IP와 Port를 연결하고, listen()으로 Client 연결 요청을 대기한다. 반복문 안에서 accept()를 사용하여 Client 접속을 수락한다.

Client로부터 HTTP Request 전체를 수신하고 Method, Path, Version, Header, Body로 파싱한다. 이후 Method와 Path에 맞는 Response를 생성한 뒤 sendall()을 사용하여 Client에게 전송한다.

Server 터미널에는 Client 주소, 원본 Request, Parsed method/path/version, Header count, Body length, Response status가 출력된다. KeyboardInterrupt 예외를 처리하여 사용자가 Ctrl + C를 누르면 서버가 안전하게 종료되도록 하였다.

9-14. server.py 전체 요약

server.py는 TCP socket을 직접 사용하여 HTTP Request를 수신하고, Request Line과 Header, Body를 직접 파싱한다. 이후 Method와 Path에 따라 GET, HEAD, POST, PUT, DELETE 요청을 분기 처리하고, 결과에 맞는 HTTP Response를 직접 구성하여 Client에게 전송한다.

특히 POST, PUT, DELETE 요청에서는 서버 폴더 안의 파일을 실제로 생성, 추가, 덮어쓰기, 삭제하도록 구현하였다. 따라서 이 서버는 단순 문자열 응답 프로그램이 아니라 TCP 소켓 통신, HTTP 메시지 처리, 파일 제어 기능을 함께 수행하는 프로그램이다.

10. client.py 코드 설명

10-1. 기본 설정 (Line 1~6)

```
1 import socket
2
3
4 HOST = "127.0.0.1"
5 PORT = 8080
6 BUFFER_SIZE = 4096
```

line 1 : TCP 소켓 통신을 사용하기 위해 socket 모듈을 import한다.

line 4~6 : Client가 접속할 Server 주소를 127.0.0.1, Port를 8080으로 설정하고, Response 수신에 사용할 BUFFER_SIZE를 지정한다.

10-2. HTTP Request 메시지 생성 (Line 9~34)

build_request() 함수는 사용자가 입력한 Method, Path, Body를 바탕으로 HTTP/1.1 Request 문자열을 직접 구성한다.

```
19 method = method.upper()
20 body_bytes = body.encode("utf-8")
```

line 19~20 : Method를 대문자로 변환하고 Body를 UTF-8 bytes로 변환한다. 이때 Body 길이를 계산하여 Content-Length Header에 사용할 수 있다.

```

22 request_lines = [
23     f"{method} {path} HTTP/1.1",
24     f"Host: {HOST}:{PORT}",
25     "User-Agent: PythonSocketHTTPClient/1.0",
26     "Content-Type: text/plain; charset=utf-8",
27     f"Content-Length: {len(body_bytes)}",
28     "Connection: close",
29     "",
30     "",
31 ]

```

line 22~31 : Request Line과 Header를 직접 작성한다. Request Line은 METHOD PATH HTTP/1.1 형식이며, Header에는 Host, User-Agent, Content-Type, Content-Length, Connection이 포함된다.

```

33 request_header = "\r\n".join(request_lines).encode("utf-8")
34 return request_header + body_bytes

```

line 33~34 : Header 문자열을 CRLF 기준으로 합친 뒤 bytes로 변환하고, Body bytes를 뒤에 붙여 최종 HTTP Request 메시지를 만든다.

10-3. TCP 연결, Request 전송, Response 수신 (Line 37~52)

send_request() 함수는 TCP 소켓으로 Server에 연결하여 Request를 보내고 Response 전체를 수신한다.

```

42 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:

```

line 42 : socket.AF_INET, socket.SOCK_STREAM을 사용하여 IPv4 기반 TCP 소켓을 생성한다.

```

43 client_socket.connect((HOST, PORT))
44 client_socket.sendall(request_bytes)

```

line 43~44 : connect()로 127.0.0.1:8080 서버에 접속하고, sendall()로 Request 전체를 전송한다.

```

46 while True:
47     chunk = client_socket.recv(BUFFER_SIZE)
48     if not chunk:
49         break
50     response += chunk

```

line 46~50 : recv()를 반복 실행하여 Server가 보낸 Response를 끝까지 수신한다. 서버가 연결을 닫으면 반복을 종료한다.

10-4. 사용자 입력 및 결과 출력 (Line 55~84)

main() 함수는 사용자에게 Method와 Path를 입력받고, POST 또는 PUT 요청이면 Body 내용도 추가로 입력받는다.

```

62 method = input("\nMethod 입력: ").strip().upper()
63 path = input("Path 입력: ").strip()
64
65 if not path.startswith("/"):
66     path = "/" + path

```

line 62~66 : 사용자가 입력한 Method와 Path를 정리한다. Path 앞에 /가 없으면 자동으로 붙여 HTTP Request Line 형식에 맞춘다.

```

68 body = ""
69 if method in ("POST", "PUT"):
70     body = input("Body 입력: ")

```

line 68~70 : POST와 PUT은 파일 생성, append, 덮어쓰기 동작에 사용할 Body를 입력받는다.

```

72 request_bytes = build_request(method, path, body)
73 response_bytes = send_request(request_bytes)
74
75 print("\n" + "=" * 70)
76 print("[CLIENT] Sent Request")
77 print(request_bytes.decode("utf-8", errors="replace"))
78 print("=" * 70)
79 print("[CLIENT] Received Response")
80 print(response_bytes.decode("utf-8", errors="replace"))

```

line 72~80 : build_request()로 Request 메시지를 만들고 send_request()로 서버에 전송한 뒤, Client 터미널에 Sent Request와 Received Response를 모두 출력한다.

10-5. client.py 전체 요약

client.py는 사용자의 입력을 받아 HTTP Request 메시지를 직접 만들고, TCP socket으로 Server에 전송하는 프로그램이다. GET, HEAD, POST, PUT, DELETE, PATCH 요청을 모두 입력할 수 있으며, POST와 PUT은 Body를 포함하여 전송할 수 있다.

11. 실행 방법

터미널 1에서 프로젝트 폴더로 이동한 뒤 python3 server.py를 실행한다.

터미널 2에서 같은 폴더로 이동한 뒤 python3 client.py를 실행한다.

Server가 먼저 실행되어 있어야 Client가 127.0.0.1:8080으로 접속할 수 있다.

12. Client / WireShark 실행 결과

12-1. GET /index.html -> 200 OK

```

socket_http_project -- ssh - 120x44
park@pangonjuil-MacBookPro: socket_http_project % cd /Users/park/Documents/Codes/2020-07-06/server-client-codes-text-1
ln -s /Users/park/Documents/Codes/2020-07-06/server-client-codes-text-1
python3 client.py
[CLIENT] TCP Socket HTTP Client
[CLIENT] Server: 127.0.0.1:8080
[CLIENT] Example methods: GET, HEAD, POST, PUT, DELETE, PATCH
[CLIENT] Example paths: /index.html, /notfound.html, /post.txt, /put.txt, /readonly.txt

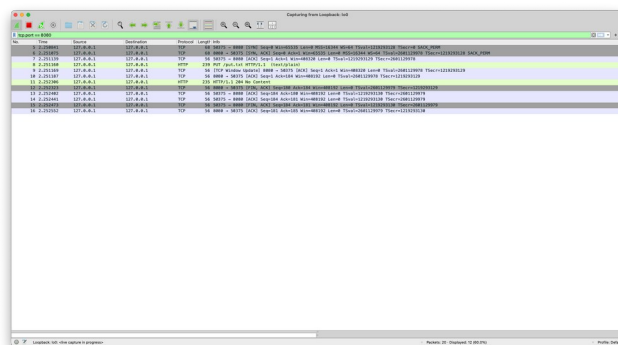
Method 입력: GET
Path 입력: /index.html

[CLIENT] Sent Request
GET /index.html HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: PythonSocketHTTPClient/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

[CLIENT] Received Response
HTTP/1.1 200 OK
Date: Mon, 06 Jul 2020 18:28:21 GMT
Server: PythonSocketHTTPServer/1.0
Content-Type: text/html; charset=utf-8
Content-Length: 486
Connection: close

<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<title>TCP Socket HTTP Project</title>
</head>
<body>
<div>TCP Socket HTTP Project</div>
<p>이 파일은 GET /index.html 요청에 대해 200 OK를 응답하기 위한 텍스트 페이지입니다. </p>
<p>Copyright © Client: Python socket 모듈을 사용하여 HTTP 메시지 전송 모듈을 작성했습니다. </p>
</body>
</html>
park@pangonjuil-MacBookPro: socket_http_project %

```



GET /index.html은 index.html 파일을 읽어 200 OK와 HTML Body를 반환한다.

12-2. GET /notfound.html -> 404 Not Found

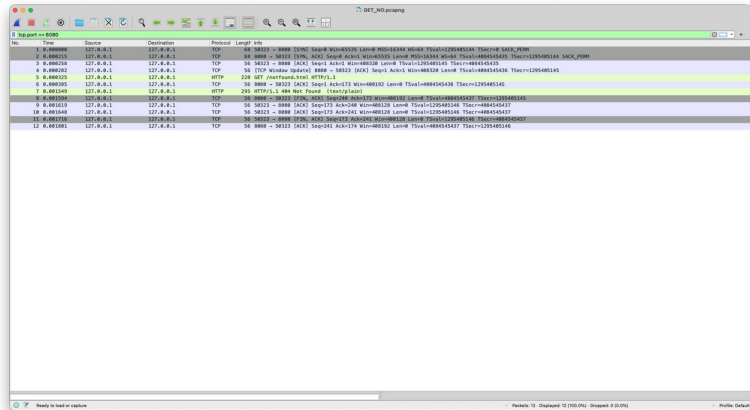
```
park@pangdongju-MacBookPro:~$ ssh -t20x44
park@pangdongju-MacBookPro:~$ socket_http_project % cd /Users/park/Documents/Codes/2020-07-06/server-client-codes-text-1
park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$ python client.py
python client.py
[CLIENT] TDP Socket HTTP Client
[CLIENT] Server: 127.0.0.1:8080
[CLIENT] Example methods: GET, HEAD, POST, PUT, DELETE, PATCH
[CLIENT] Example paths: /index.html, /notfound.html, /post.txt, /put.txt, /readonly.txt

Method 입력: GET
Path 입력: /notfound.html

[CLIENT] Sent Request
GET /notfound.html HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: PythonSocketHTTPClient/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

[CLIENT] Received Response
HTTP/1.1 404 Not Found
Date: Mon, 06 Jul 2020 18:31:11 GMT
Server: PythonSocketHTTPServer/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

404 Not Found: /notfound.html does not exist on this server.
park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$
```



GET /notfound.html은 파일이 없으므로 404 Not Found를 반환한다.

POST 또는 PUT 후 GET /post.txt, GET /put.txt를 실행하면 파일 제어 결과가 실제로 반영되었는지 확인할 수 있다.

12-3. HEAD /index.html -> 200 OK

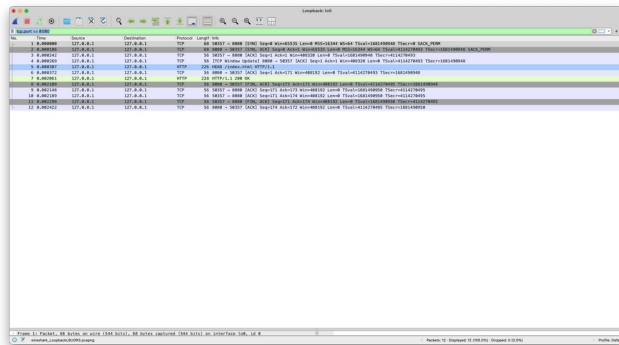
```
park@pangdongju-MacBookPro:~$ ssh -t20x44
park@pangdongju-MacBookPro:~$ socket_http_project % cd /Users/park/Documents/Codes/2020-07-06/server-client-codes-text-1
park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$ python client.py
python client.py
[CLIENT] TDP Socket HTTP Client
[CLIENT] Server: 127.0.0.1:8080
[CLIENT] Example methods: GET, HEAD, POST, PUT, DELETE, PATCH
[CLIENT] Example paths: /index.html, /notfound.html, /post.txt, /put.txt, /readonly.txt

Method 입력: HEAD
Path 입력: /index.html

[CLIENT] Sent Request
HEAD /index.html HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: PythonSocketHTTPClient/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

[CLIENT] Received Response
HTTP/1.1 200 OK
Date: Mon, 06 Jul 2020 18:32:16 GMT
Server: PythonSocketHTTPServer/1.0
Content-Type: text/html; charset=utf-8
Content-Length: 406
Connection: close

park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$
```



HEAD /index.html은 index.html 파일이 존재하므로 200 OK를 반환한다. 단, HEAD Method의 특성상 Response Header만 전송하고 Body는 전송하지 않는다.

12-4. HEAD /missing.html -> 404 Not Found

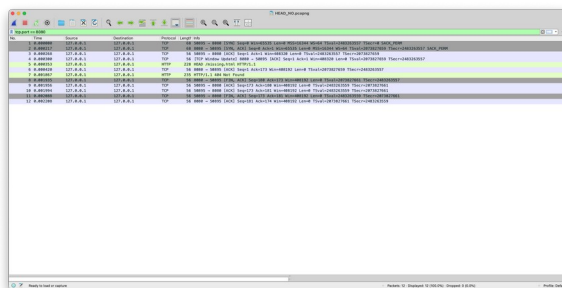
```
park@pangdongju-MacBookPro:~$ ssh -t20x44
park@pangdongju-MacBookPro:~$ socket_http_project % cd /Users/park/Documents/Codes/2020-07-06/server-client-codes-text-1
park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$ python client.py
python client.py
[CLIENT] TDP Socket HTTP Client
[CLIENT] Server: 127.0.0.1:8080
[CLIENT] Example methods: GET, HEAD, POST, PUT, DELETE, PATCH
[CLIENT] Example paths: /index.html, /notfound.html, /post.txt, /put.txt, /readonly.txt

Method 입력: HEAD
Path 입력: /missing.html

[CLIENT] Sent Request
HEAD /missing.html HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: PythonSocketHTTPClient/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

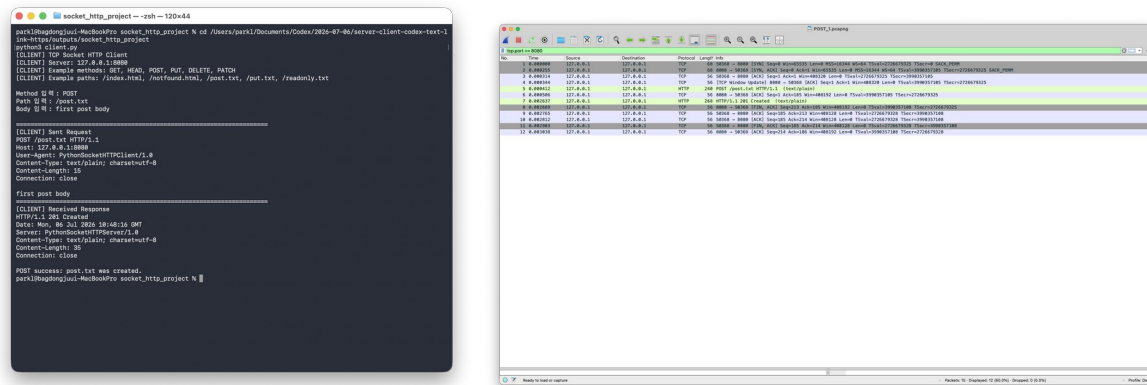
[CLIENT] Received Response
HTTP/1.1 404 Not Found
Date: Mon, 06 Jul 2020 18:33:16 GMT
Server: PythonSocketHTTPServer/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: close

park@pangdongju-MacBookPro:~/Documents/Codes/2020-07-06/server-client-codes-text-1$
```



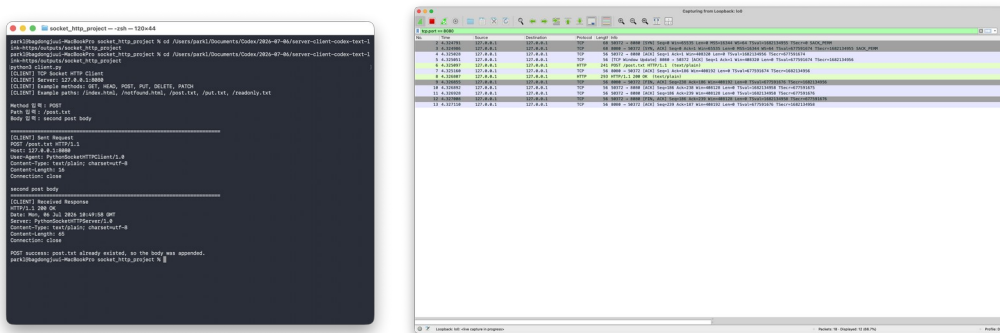
HEAD /missing.html은 요청한 파일이 존재하지 않으므로 404 Not Found를 반환한다. HEAD Method 이므로 오류 응답에서도 Body 없이 Header만 반환한다.

12-5. POST /post.txt -> 201 Created



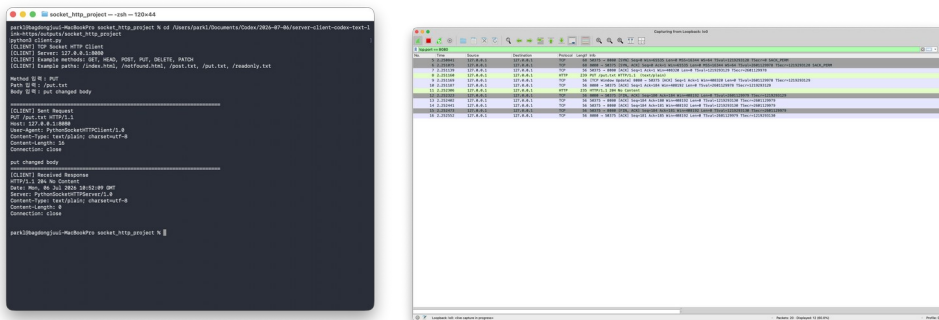
첫 번째 POST /post.txt는 post.txt가 없으면 파일을 생성하고 201 Created를 반환한다.

12-6. POST /post.txt -> 200 OK



두 번째 POST /post.txt는 파일이 이미 존재하므로 기존 내용 뒤에 새로운 Body를 append하고 200 OK를 반환한다. 이후 GET /post.txt를 실행하면 두 줄의 내용이 모두 출력되어 append 결과를 확인할 수 있다.

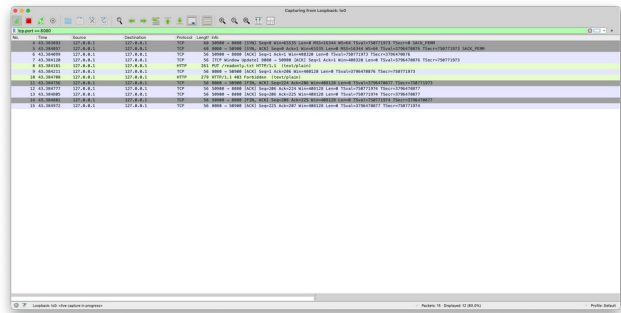
12-7. PUT /put.txt -> 204 No Content



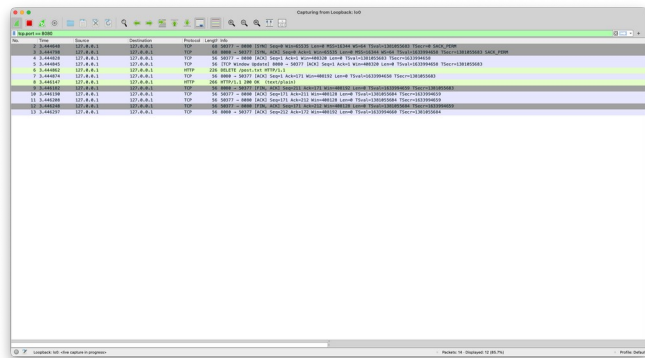
PUT /put.txt는 Body 내용으로 put.txt를 덮어쓴다.

응답은 204 No Content이다. 이것은 실패가 아니라 요청은 성공했지만 응답 Body가 없다는 뜻이다.

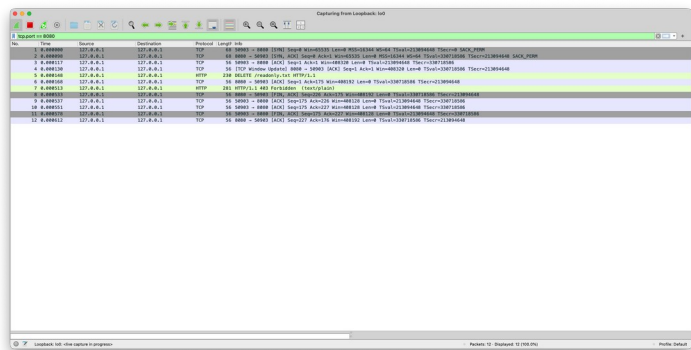
12-8. PUT /readonly.txt -> 403 Forbidden



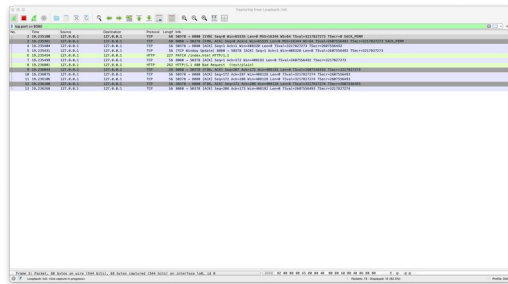
12-9. DELETE /post.txt -> 200 OK



12-7. DELETE /readonly.txt -> 403 Forbidden



12-8. PATCH /index.html -> 400 Bad Request



13. Server 실행 결과

각 요청 처리 후 Server는 Response status를 출력한다. 예를 들어 GET 성공은 HTTP/1.1 200 OK, POST 생성은 HTTP/1.1 201 Created, PUT 성공은 HTTP/1.1 204 No Content, 잘못된 Method는 HTTP/1.1 400 Bad Request로 출력된다.

14. 전체 실행 결과 요약

총 11개의 Method-Response case를 구현하여 과제에서 요구한 5개 이상의 case 조건을 충족한다.